PROJECT REPORT

Fundamentals of AI and ML – Evaluated Course Project

STUDENT PERFORMANCE PREDICTOR USING MACHINE LEARNING

Submitted By: Adarsh Tiwari

Roll No: 25BAI10019

INDEX

SERIAL NUMBER	TOPIC
1	INTRODUCTION
2	PROBLEM STATEMENT
3	DATASET DESCRIPTION
4	PROCEDURE
5	IMPLEMENTATION DIAGRAM
6	CODE
7	RESULT
8	CHALLANGES FACED
9	LEARNING OUTCOMES
10	FUTURE IMPROVEMENTS

INTRODUCTION

In today's education system, students often struggle to understand their academic performance until final results are announced. This delay makes it difficult for them to identify weaknesses and improve in time. Predicting student performance in advance can help students take corrective actions and improve their outcomes.

The problem addressed in this project is the lack of early insight into a student's expected performance. Factors such as study hours, attendance, and previous academic records play an important role in determining final results, but students and educators do not always analyze these factors effectively.

This problem is important because early prediction can support better decision-making. Students can adjust their study habits, improve attendance, and focus on weak areas. Teachers can also identify students who may need extra support.

In real life, this issue is commonly seen in schools and colleges, where students only realize their performance level after exams. By applying Machine Learning techniques, this project provides a simple and effective way to estimate student performance beforehand, making education more proactive and data-driven.

PROBLEM STATEMENT

The objective of this project is to develop a Machine Learning model that can predict a student's final academic score based on key factors such as study hours, attendance, and previous marks.

In many educational settings, students do not have a clear understanding of their expected performance until final results are declared. This lack of early insight makes it difficult for them to take timely actions to improve their outcomes. Additionally, educators may find it challenging to identify students who are at risk of underperforming.

This project aims to address this issue by using historical academic data to build a predictive model. The system analyzes important features influencing performance and provides an estimated final score, enabling students to make informed decisions and improve their academic progress.

Dataset Description

In this project, a custom dataset was created manually in CSV format to train the machine learning model. The dataset was designed using simple and realistic values to represent the relationship between study habits and student performance.

The file, named ***customstudentdataset.csv,*** contains data related to key factors that influence academic results. Instead of using a complex or external dataset, a small and structured dataset was prepared to clearly demonstrate the working of the model.

The dataset includes the following features:

COLUMN NAME	DESCRIPTION
Study_Hours	Number of hours studied per day
Attendance	Attendance percentage (%)
Previous_Marks	Marks obtained in previous exams
Final_Score	Final score (target variable)

Although the dataset is simple and limited in size, it is sufficient to demonstrate how machine learning models can be trained and used for prediction tasks. The dataset can also be expanded in the future to improve accuracy and make the model more robust.

Procedure

1.Data Collection :

A custom dataset was created manually in CSV format (customstudentdataset.csv).

2. Data Preprocessing :

The dataset was loaded using the Pandas library. The data was already clean and structured, so no major preprocessing steps like handling missing values were required. The input features and target variable were separated.

3. Splitting the Dataset

The dataset was divided into training and testing sets using the train_test_split function from Scikit-learn. 70% of the data was used for training , 30% was used for testing. This helps evaluate how well the model performs on unseen data.

4. Model Selection

A Linear Regression algorithm was selected for this project. This algorithm is suitable because:

- It predicts continuous numerical values
- It is simple and easy to interpret
- It works well for small datasets

5. Model Training

The Linear Regression model was trained using the training dataset. During training, the model learns the relationship between input features (study hours, attendance, previous marks) and the output (final score).

6. Prediction

After training, the model was used to:

- Predict final scores for the test dataset
- Predict scores for custom input values (user-defined data)

This demonstrates how the model can be used in real-life scenarios.

7. Model Evaluation

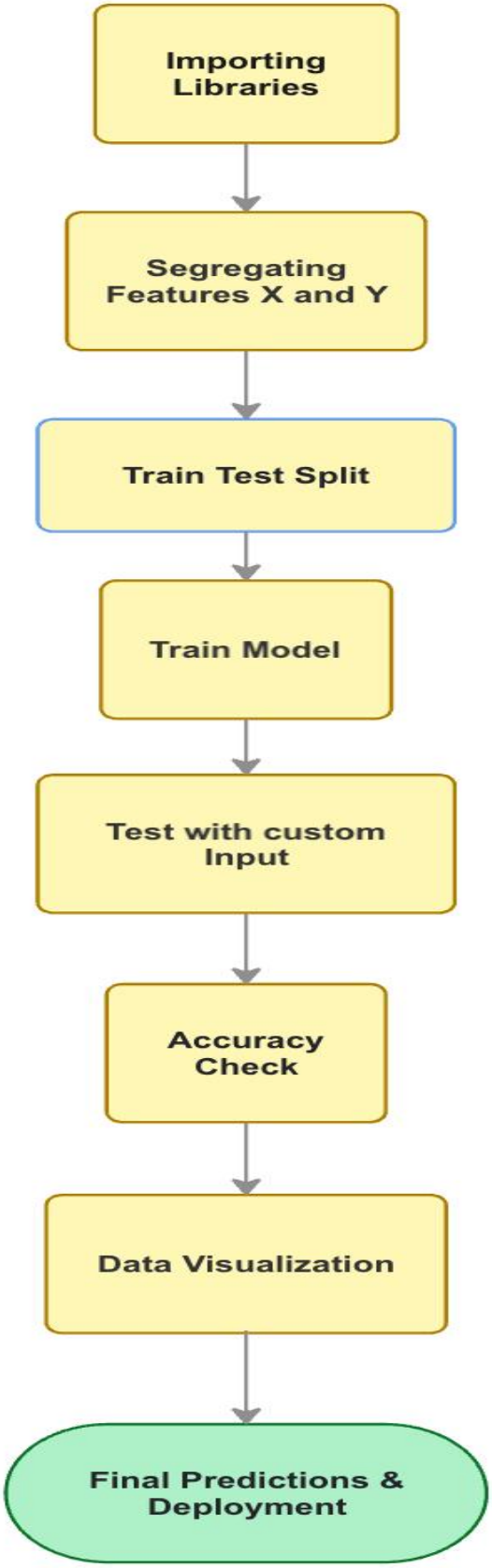
The performance of the model was evaluated using the R^2 (R-squared) score.

- The R^2 score measures how well the model predicts the output
- A value closer to 1 indicates better accuracy.

8. Data Visualization

A scatter plot was created using Matplotlib to visualize the relationship between study hours and final score. This helps in understanding how the input variable affects the output.

IMPLEMENTATION DIAGRAM



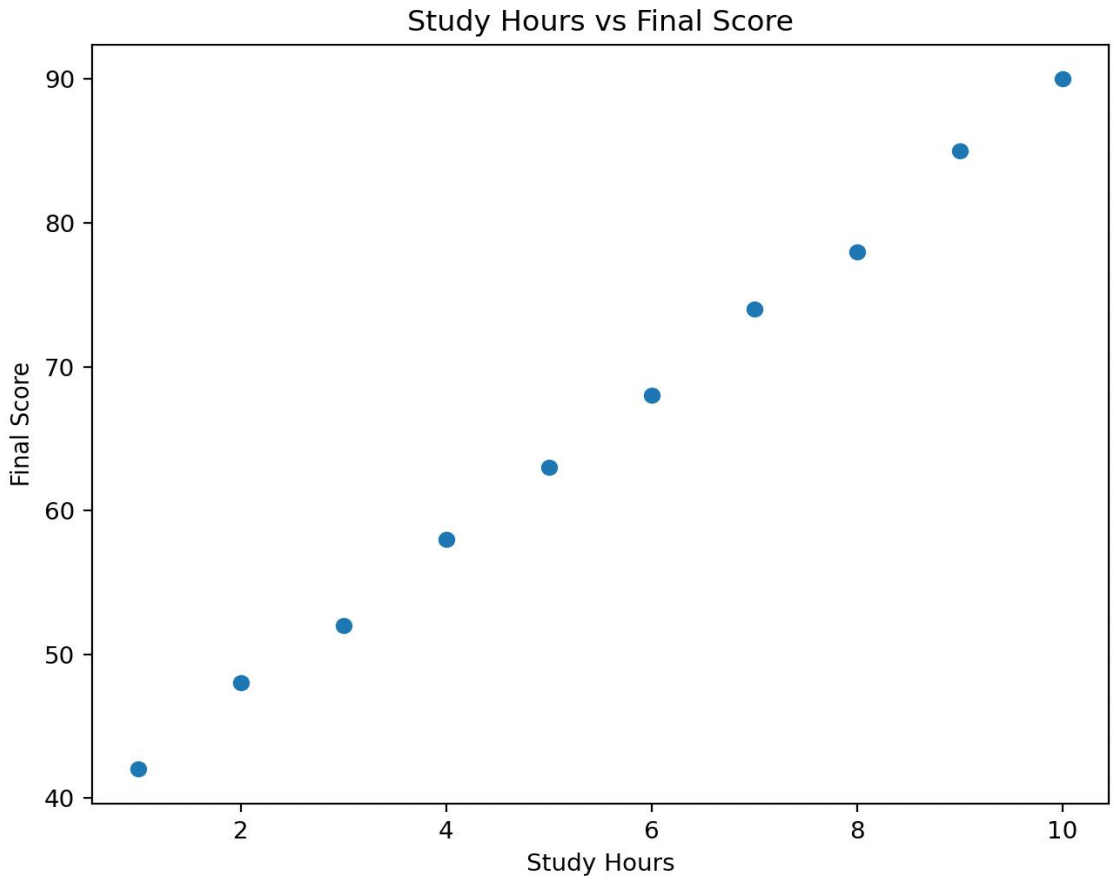
CODE

student-performance.py > ...

```python
from sklearn.metrics import r2_score
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("customstudentdataset.csv")

# Step 2 :

X = df[['Study_Hours', 'Attendance', 'Previous_Marks']]
y = df['Final_Score']

# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Step 4: Train Model

model = LinearRegression()
model.fit(X_train, y_train)


# Step 5: Predictions

predictions = model.predict(X_test)
print("Predictions:", predictions)
print("Actual:", list(y_test))

# Step 6: Test with Custom Input

custom_input = pd.DataFrame([[6, 80, 70]], columns=['Study_Hours', 'Attendance', 'Previous_Marks'])
predicted_score = model.predict(custom_input)
print("\nPredicted Final Score:", predicted_score[0])


# Step 7 : Accuracy check

accuracy = r2_score(y_test, predictions)
print("Model Accuracy (R2 Score):", accuracy)


# Step 8: Visualization

plt.scatter(df['Study_Hours'], df['Final_Score'])
plt.xlabel("Study Hours")
plt.ylabel("Final Score")
plt.title("Study Hours vs Final Score")
plt.show()
```

RESULT

```
python -u "/Users/adarshtiwari/Desktop/AI-ML-PROJECT/student-performance.py"
(base) adarshtiwari@MAC AI-ML-PROJECT % python -u "/Users/adarshtiwari/Deskto
.py"
Predictions: [47.03942871 84.49726868 79.14610291]
Actual: [48, 85, 78]

Predicted Final Score: 1972813325.7249908
Model Accuracy (R2 Score): 0.9967787042438961
```



CHALLANGES FACED

- **Creating a Realistic Dataset** :

Since the dataset was created manually, it was challenging to ensure that the values were realistic and followed a logical relationship between input features and the final score.

- **Understanding Machine Learning Concepts :**

Gaining a clear understanding of how Linear Regression works and how the model learns from data required careful study and practice.

- **Selecting Relevant Features :**

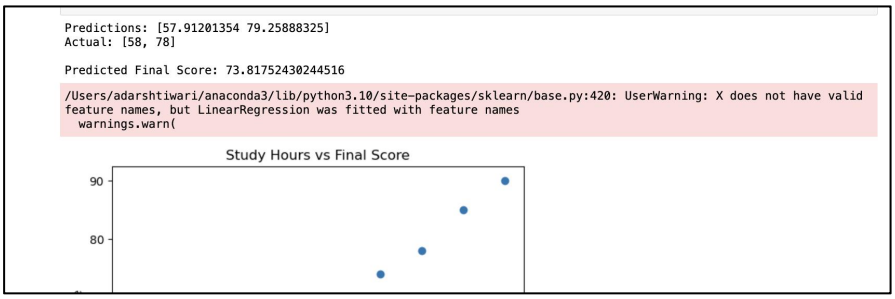
Deciding which factors (study hours, attendance, previous marks) should be included in the model was an important step to ensure meaningful predictions.

- **Handling Train-Test Split**

Understanding how to properly divide the dataset into training and testing sets to avoid overfitting was initially confusing.

- **Fixing Prediction Warning**

While making predictions, a warning related to feature names appeared. This was resolved by ensuring that the input data format matched the training data format.



LEARNING OUTCOMES

1. **Understanding of Machine Learning Basics**
Gained knowledge about how machine learning models work, especially supervised learning and prediction tasks.
2. **Knowledge of Linear Regression**
Learned how Linear Regression is used to predict continuous values and how it establishes relationships between input features and output.
3. **Working with Datasets (CSV Files)**
Learned how to create, manage, and use CSV datasets for training machine learning models.
4. **Data Handling using Pandas**
Developed skills in loading, analyzing, and manipulating data using the Pandas library.
5. **Model Training and Testing**
Understood how to split data into training and testing sets and why it is important for evaluating model performance.
6. **Model Evaluation using R^2 Score**
Learned how to measure the accuracy of a model and interpret evaluation metrics.
7. **Making Predictions**
Gained practical experience in using a trained model to make predictions on new input data.
8. **Data Visualization**
Learned how to visualize data using Matplotlib to better understand relationships between variables.
9. **Problem-Solving Skills**
Improved the ability to break down a real-world problem and solve it using a structured machine learning approach.

FUTURE IMPROVEMENTS

- **Use a Larger Dataset :**

The current dataset is small and manually created. Using a larger and real-world dataset can improve the model's accuracy and reliability.

- **Add More Features :**

Additional factors such as sleep hours, assignment scores, participation, and stress levels can be included to make predictions more comprehensive.

- **Use Advanced Machine Learning Models :**

More powerful algorithms like Decision Trees, Random Forest, or Neural Networks can be used to achieve better performance.

- **Build a User Interface (GUI/Web App) :**

The project can be converted into a user-friendly application using tools like Tkinter (GUI) or Flask (web app).

- **Model Optimization :**

Techniques like hyperparameter tuning and cross-validation can be applied to further improve model performance.